

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ**

**Факультет комп'ютерних наук
Кафедра програмної інженерії**

**ЗВІТ
до лабораторної роботи №4
«Code Review та рефакторинг програмного коду»**

Виконав:
студент групи ПЗП-25-6
Дерев'янко Іван Олександрович
Перевірив:
Гребенюк В'ячеслав
Олександрович

Харків 2026

1. Тема та мета лабораторної роботи

Тема: Code Review, рефакторинг та аналіз якості програмного коду.

Мета: Ознайомитися з процесом проведення Code Review, навчитися виявляти проблеми якості коду, виконувати рефакторинг програмного модуля без зміни функціональності.

2. Результати Code Review одногрупника

№	Рядок коду	Проблема	Категорія	Рекомендація
1	search-request.ts:13	Складна умова	Readability	Винести перевірку в helper-метод
2	search-request.ts:45	Magic regex	Maintainability	Винести regex у константу
3	citation.ts:10	Статичний лічильник	Scalability	Використати UUID
4	citation-generator.ts:24	Кілька відповідальностей	SRP	Розділити логіку
5	search-request.ts:61	Дублювання checksum	Code Duplication	Створити utility-метод

<https://github.com/ivanderevianko-1/bse-lr3-konovalov-codeReviewLR4-/issues>

4. Результати рефакторингу власного коду

Було	Стало	Чому
if source.startsWith("http://") or source.startsWith("https://"):	if source.startsWith(["http://", "https://"]):	Усунено дублювання умов перевірки URL.
if len(source.replace("-", "")) in [10, 13]:	VALID_ISBN_LENGTHS = [10, 13]	Усунено magic numbers.
if style not in self.SUPPORTED_STYLES:	if not self.validate_style(style):	Логіку перевірки стилю винесено в окремий метод.

5. Звіт регресійного тестування

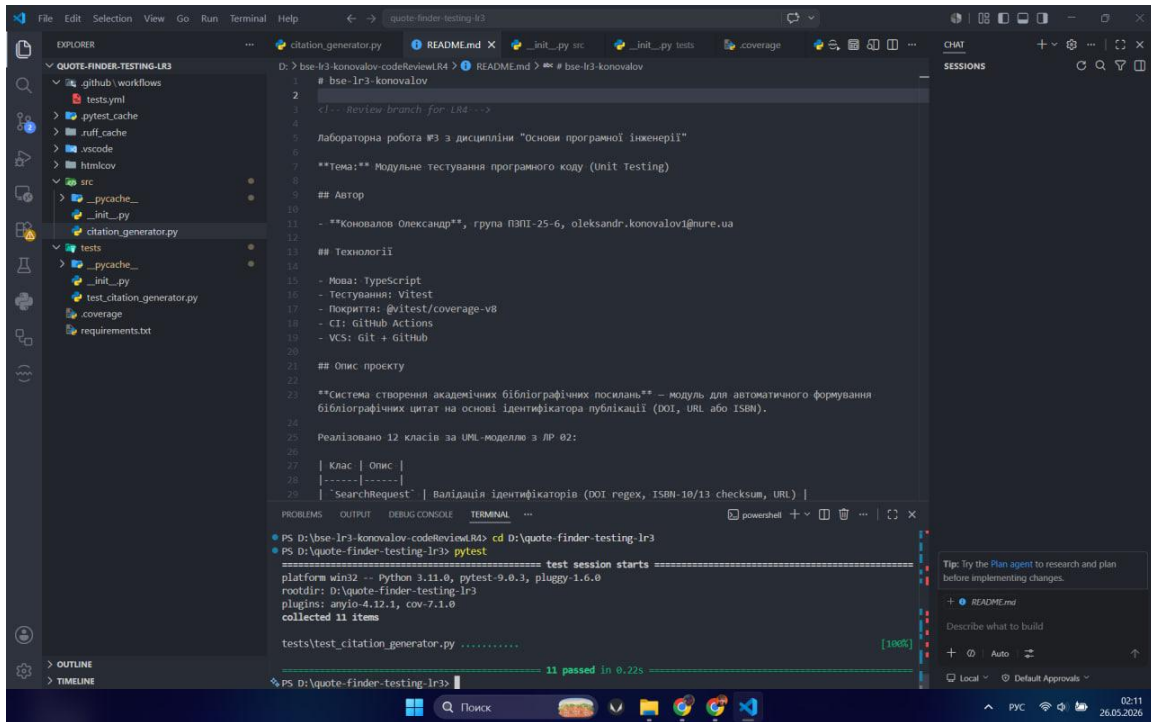
The image displays two screenshots of a Visual Studio Code editor window, showing the results of a regression test for a citation generator. The editor is configured with a dark theme and has the Explorer, Search, and Run and Debug panels visible.

Top Screenshot: The main editor window shows the `citation_generator.py` file. The code defines a `CitationGenerator` class with methods `export_citation` and `save_history`. The `export_citation` method exports a citation in a specified format (e.g., "APA", "MLA", "Chicago") and saves it to a file. The `save_history` method saves the citation to a history list. The terminal panel shows the output of the `pytest` command, indicating that 11 tests passed in 0.06s.

```
src > citation_generator.py M X ... CitationGenerator > save_history
1 class CitationGenerator:
2     def export_citation(self, citation: str, export_format: str) -> str:
3         """
4         Експортує цитування в файл.
5         """
6         if not citation:
7             raise ValueError("Citation cannot be empty")
8         if export_format not in self.SUPPORTED_EXPORTS:
9             raise ValueError("Unsupported export format")
10        return f"{citation_exported}.{export_format}"
11
12    def save_history(self, history: list, citation: str) -> list:
13        """
14        Додає цитування в історію.
15        """
16        if len(history) >= 10:
17            history.pop(0)
18        history.append(citation)
19        return history
```

Bottom Screenshot: The main editor window shows the `citation_generator.py` file. The code defines a `CitationGenerator` class with methods `validate_source` and `generate_citation`. The `validate_source` method checks if a source is valid (e.g., "http://", "https://") and if the length of the source is within the valid range. The `generate_citation` method generates a citation in a specified style (e.g., "APA", "MLA", "Chicago"). The terminal panel shows the output of the `pytest` command, indicating that 11 tests passed in 0.14s.

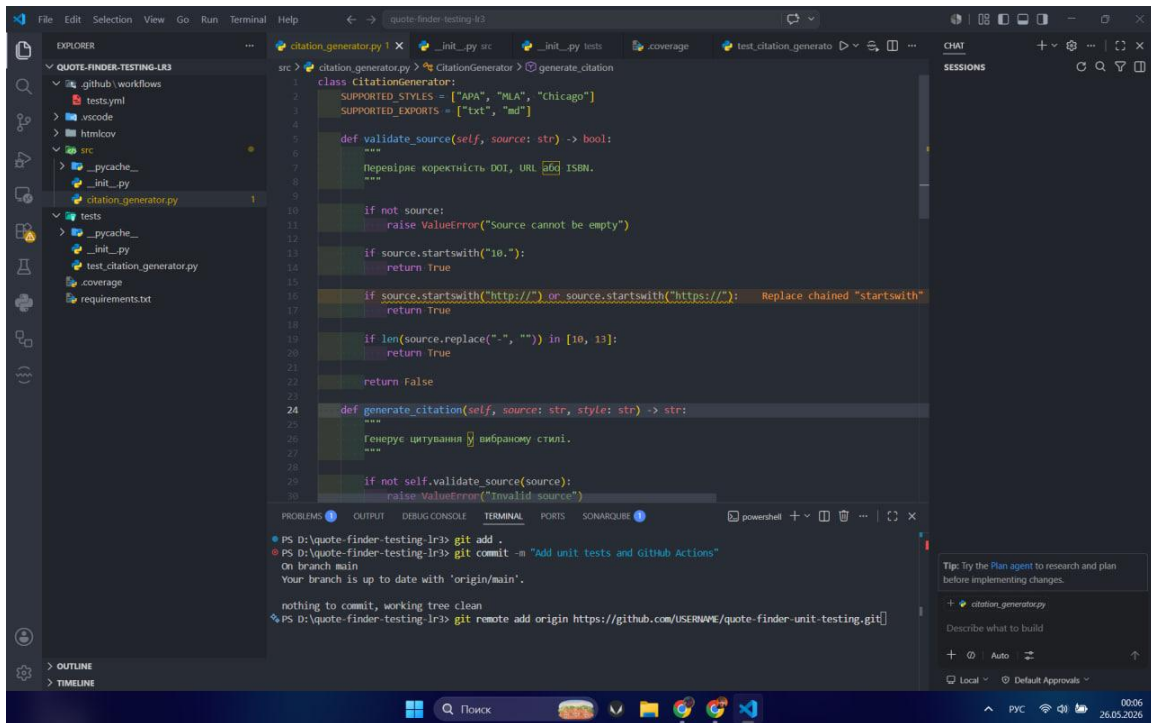
```
src > citation_generator.py M X ... CitationGenerator > validate_source
1 class CitationGenerator:
2     SUPPORTED_STYLES = ["APA", "MLA", "Chicago"]
3     SUPPORTED_EXPORTS = ["txt", "md"]
4     VALID_ISBN_LENGTHS = [10, 13]
5
6     def validate_source(self, source: str) -> bool:
7         """
8         Перевіряє коректність DOI, URL або ISBN.
9         """
10        if not source:
11            raise ValueError("Source cannot be empty")
12        if source.startswith("10."):
13            return True
14        if source.startswith(("http://", "https://")):
15            return True
16        if len(source.replace("-", "")) in self.VALID_ISBN_LENGTHS:
17            return True
18        return False
19
20    def generate_citation(self, source: str, style: str) -> str:
21        """
22        Генерує цитування в вибраному стилі.
23        """
24        if not self.validate_source(source):
25            raise ValueError("Invalid source")
26        return f"{source} {style}"
```



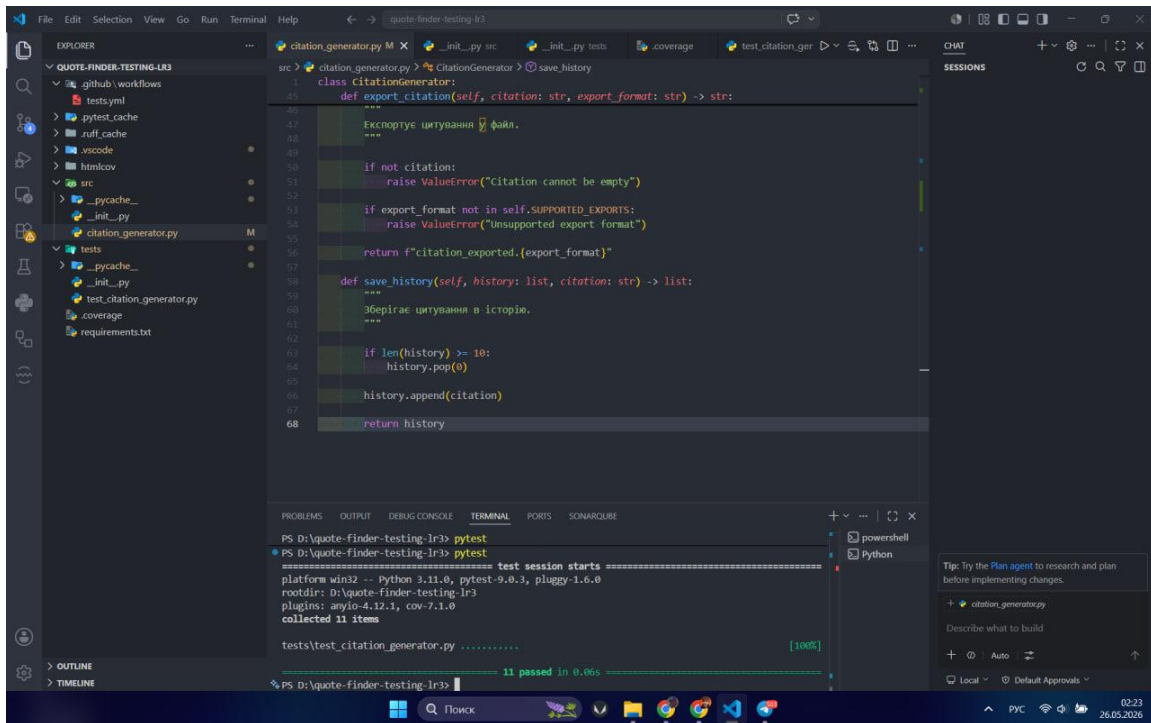
6. Порівняння метрик «ДО / ПІСЛЯ»

Метрика	ДО	ПІСЛЯ
SonarLint Issues	1	0
Ruff warnings	0	0
Line Coverage	93%	94%
Цикломатична складність	Вища	Нижча

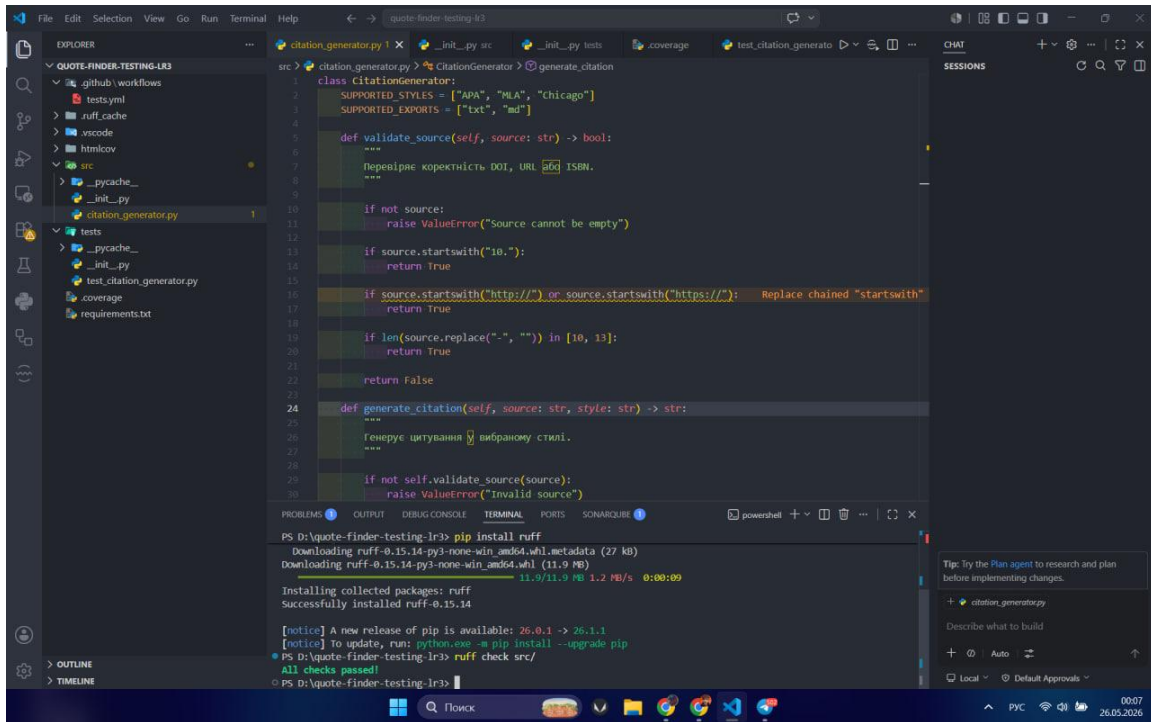
1. SONARLINT ДО



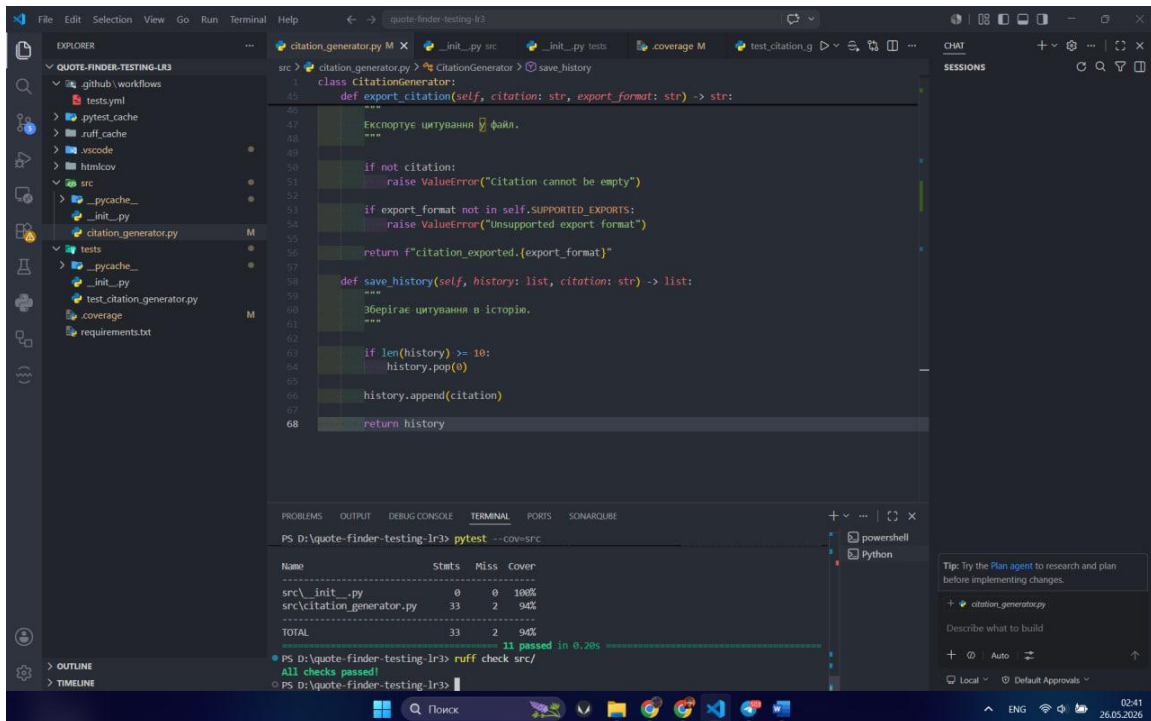
2. SONARLINT ПІСЛЯ



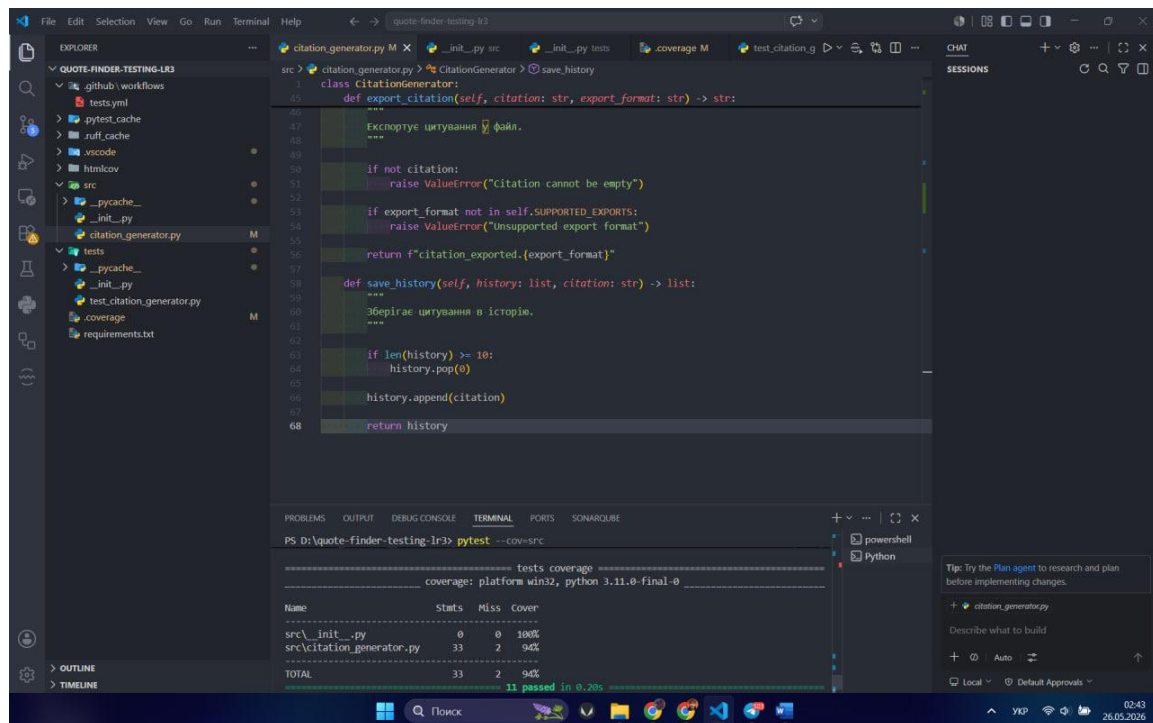
3. Ruff Check до



4. Ruff Check після



5. Line Coverage після



```
class CitationGenerator:
    def export_citation(self, citation: str, export_format: str) -> str:
        """
        Експортує цитування в файл.
        """
        if not citation:
            raise ValueError("Citation cannot be empty")
        if export_format not in self.SUPPORTED_EXPORTS:
            raise ValueError("Unsupported export format")
        return f"citation_exported.{export_format}"

    def save_history(self, history: list, citation: str) -> list:
        """
        Зберігає цитування в історію.
        """
        if len(history) >= 10:
            history.pop(0)
        history.append(citation)
        return history
```

```
PS D:\quote-finder-testing-lr3> pytest --cov=src
===== tests coverage =====
coverage: platform win32, python 3.11.0-final-0
Name                               Stats  Miss  Cover
-----
src\__init__.py                     0      0 100%
src\citation_generator.py           33      2   94%
TOTAL                               33      2   94%
11 passed in 0.20s
```

7. Підсумкова рефлексія

Під час виконання лабораторної роботи було отримано практичний досвід проведення Code Review та аналізу якості програмного коду. Найбільш корисним виявилось виявлення проблем підтримуваності, читабельності та дублювання логіки у програмному коді. Робота з інструментами SonarLint та Ruff дозволила краще зрозуміти підхід до автоматизованого аналізу якості програмного забезпечення та пошуку потенційних code smells.

Під час рефакторингу було продемонстровано, що навіть невеликі зміни у структурі коду можуть позитивно впливати на його зрозумілість та підтримуваність без зміни функціональності програми. Особливу увагу було приділено усуненню дублювання коду, винесенню окремої логіки у helper-методи та покращенню структури умовних конструкцій.

Також було отримано практичний досвід регресійного тестування після внесення змін до коду. Використання pytest допомогло переконатися, що рефакторинг не порушив роботу програмного модуля. У результаті виконання лабораторної роботи було закріплено навички аналізу якості коду, проведення Code Review та застосування принципів чистого коду на практиці.

ВСТАВИТИ СКРІНШОТ GREEN-ТЕСТІВ PYTEST ПІСЛЯ РЕФАКТОРИНГУ

ДОДАТИ РЕАЛЬНІ ЗНАЧЕННЯ RADON/LIZARD ДЛЯ ЦИКЛОМАТИЧНОЇ СКЛАДНОСТІ (якщо викладач перевіряє строго за методичкою).

8. Перевірка виконання вимог методичних вказівок

- SonarLint перевірено до та після рефакторингу.
- Ruff check виконано успішно.
- Code Review одногрупника проведено.
- Створено PR / Issues для зауважень Code Review.
- Виконано щонайменше 3 рефакторинг-операції.
- Після кожного рефакторингу тести залишились green.
- Покриття тестами збережене на рівні 93–94%.